

Batch – T5**Assignment No. – 9****Title – Graphs, Dynamic Programming****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011**

1)

Problem

You are given a weighted tree with N nodes and $N - 1$ edges.

E is defined as $\sum_{i=1}^N \sum_{j=i+1}^N F(i, j)$. Also, $F(i, j)$ is equal to = (Maximum value of the edge that is present on the simple path between node i and j) $\times i \times j$

You are required to determine the value of E modulo $10^9 + 7$.

Input format

- The first line contains an integer N denoting the number of nodes in a tree.
- The next $N - 1$ lines contain three space-separated integers u, v, w denoting an edge between node u and v with weight w .

Output format

Print the required value of E modulo $10^9 + 7$.

Constraints

$$1 \leq N \leq 2 \times 10^5$$

$$1 \leq w \leq 10^6$$

$$1 \leq u, v \leq N$$

Sample Input	Sample Output
<pre>3 1 2 10 1 3 2</pre>	<pre>86</pre>

Idea:

We are asked to calculate a value E for a given weighted tree where N nodes and $N-1$ edges are provided. E is defined as:

$$E = \sum_{i=1}^N \sum_{j=i+1}^N F(i, j)$$

where $F(i, j)$ is:

- The maximum edge weight on the simple path between nodes i and j ,
- Multiplied by i ,
- Multiplied by j .

We need to compute $E \bmod (10^9 + 7)$.

Code:

```
import java.util.*;

public class Q1 {

    static final int MAXN = 200005;
    static final int LOG = 20;
    static final int MOD = 1000000007;

    static class Edge {
        int to, weight;

        Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    static List<Edge>[] adj = new ArrayList[MAXN];
    static int[][] parent = new int[MAXN][LOG];
    static int[][] maxEdge = new int[MAXN][LOG];
    static int[] depth = new int[MAXN];
    static int N;

    static void dfs(int node, int par, int w) {
        parent[node][0] = par;
        maxEdge[node][0] = w;
        for (Edge e : adj[node]) {
            if (e.to != par) {
                depth[e.to] = depth[node] + 1;
                dfs(e.to, node, e.weight);
            }
        }
    }

    static void preprocess() {
        for (int j = 1; j < LOG; j++) {
            for (int i = 1; i <= N; i++) {
                if (parent[i][j - 1] != -1) {
                    parent[i][j] = parent[parent[i][j - 1]][j - 1];
                    maxEdge[i][j] = Math.max(maxEdge[i][j - 1], maxEdge[parent[i][j - 1]][j - 1]);
                }
            }
        }
    }

    static int getMaxEdge(int u, int v) {
        if (depth[u] < depth[v]) {
            int temp = u;
            u = v;
            v = temp;
        }
        while (depth[u] > depth[v]) {
            u = parent[u][1];
        }
        if (u == v) return u;
        while (parent[u][LOG - 1] != parent[v][LOG - 1]) {
            u = parent[u][LOG - 1];
            v = parent[v][LOG - 1];
        }
        return parent[u][LOG - 1];
    }
}
```

```
        u = v;
        v = temp;
    }
    int maxW = 0;
    int diff = depth[u] - depth[v];

    for (int i = LOG - 1; i >= 0; i--) {
        if ((diff & (1 << i)) != 0) {
            maxW = Math.max(maxW, maxEdge[u][i]);
            u = parent[u][i];
        }
    }

    if (u == v)
        return maxW;

    for (int i = LOG - 1; i >= 0; i--) {
        if (parent[u][i] != parent[v][i]) {
            maxW = Math.max(maxW, Math.max(maxEdge[u][i], maxEdge[v][i]));
            u = parent[u][i];
            v = parent[v][i];
        }
    }

    return Math.max(maxW, Math.max(maxEdge[u][0], maxEdge[v][0]));
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    N = sc.nextInt();

    for (int i = 1; i <= N; i++) {
        adj[i] = new ArrayList<>();
    }

    for (int i = 1; i < N; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        int w = sc.nextInt();
        adj[u].add(new Edge(v, w));
        adj[v].add(new Edge(u, w));
    }

    dfs(1, -1, 0);
    preprocess();

    long E = 0;
    for (int i = 1; i <= N; i++) {
        for (int j = i + 1; j <= N; j++) {
            int maxW = getMaxEdge(i, j);
```

```
        E = (E + 1L * maxW * i * j) % MOD;
    }
}

System.out.println(E);
}
}
```

Test Case 1:

Enter the number of nodes (N): 5

Enter 4 edges in the format (u v w) where:

u = node1, v = node2, w = weight of the edge between u and v

Edge 1: 1 2 5

Edge 2: 2 3 2 3

Edge 3: 3 4 7

Edge 4: 4 5 1

The value of E modulo $10^9 + 7$ is: 441

Test Case 2:

Enter the number of nodes (N): 3

Enter 2 edges in the format (u v w) where:

u = node1, v = node2, w = weight of the edge between u and v

Edge 1: 1 2 3 10

Edge 2: 1 3 2

The value of E modulo $10^9 + 7$ is: 86

2)

Problem

There is a country with N villages and each village has a number associated with them and M roads, a road connects two different villages

In this country, the Prime minister decided to test the hospitals to see if they can handle another wave of the coronavirus pandemic.

A village has the vaccine if the number associated with it is a prime number (villages numbered 2,3,5,7,11... will have a vaccine with them)

For each village find the minimum time required for the vaccine to arrive there if for each road between villages u and v it takes $\max(u, v)$ time to travel this road

Note: The graph does not have self-loops or multi edges

Input :

The first line will contain N and M , the number of villages and the number of roads respectively

The next M lines contain two integers u and v (meaning there is a road between villages u and v and it takes $\max(u, v)$ time to travel this road)

Output:

Print N space-separated integers which is the minimum time for the vaccine to arrive at that village, if it's impossible for the vaccine to arrive at a particular village then print -1

Constraints:

$$1 \leq N \leq 2 * 10^5$$

$$0 \leq M \leq \min(2 * 10^5, (N * (N - 1))/2)$$

$$1 \leq u, v \leq N$$

Sample Input	Sample Output
6 4 1 2 1 3 1 4 2 5	2 0 0 6 0 -1

Algorithm:

1. Identify Primes for Vaccine Villages:

- Precompute the prime numbers up to NNN using the Sieve of Eratosthenes. Mark these villages as starting points since they have the vaccine.
2. Graph Representation:
 - Use an adjacency list to represent the graph, where each edge weight is the $\max(u, v)$ between connected nodes uuu and vvv .
 3. Multi-Source Dijkstra's Algorithm:
 - Initialize a priority queue and add all villages with vaccines (prime-numbered villages) with a starting distance of 0.
 - For each village, store the minimum distance (time) it takes for the vaccine to reach it.
 - Use Dijkstra's algorithm to propagate the minimum time for the vaccine to reach each village:
 - Pop the village with the minimum time from the queue.
 - For each neighbor, calculate the time it would take to reach the neighbor through the current village.
 - If this new time is less than the previously recorded time for that neighbor, update the time and push the neighbor into the priority queue.
 4. Handle Unreachable Villages: For any village that wasn't updated (i.e., it is unreachable from any prime-numbered village), set its time to -1.
 5. Output the Result: Print the minimum times for each village to receive the vaccine.

Code:

```
import java.util.*;

public class Q2 {
    static final int INF = Integer.MAX_VALUE;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int M = sc.nextInt();

        // Graph representation using adjacency list
        List<int[]>[] graph = new ArrayList[N + 1];
        for (int i = 1; i <= N; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int i = 0; i < M; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int weight = Math.max(u, v);
            graph[u].add(new int[]{v, weight});
            graph[v].add(new int[]{u, weight});
        }

        // Prime detection
        boolean[] isPrime = new boolean[N + 1];
        Arrays.fill(isPrime, true);
        isPrime[0] = isPrime[1] = false;
```

```
for (int i = 2; i * i <= N; i++) {
    if (isPrime[i]) {
        for (int j = i * i; j <= N; j += i) {
            isPrime[j] = false;
        }
    }
}

// Multi-source Dijkstra's algorithm
int[] dist = new int[N + 1];
Arrays.fill(dist, INF);
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a[1]));

for (int i = 1; i <= N; i++) {
    if (isPrime[i]) {
        dist[i] = 0;
        pq.offer(new int[]{i, 0});
    }
}

while (!pq.isEmpty()) {
    int[] node = pq.poll();
    int u = node[0];
    int d = node[1];

    if (d > dist[u]) continue;

    for (int[] edge : graph[u]) {
        int v = edge[0];
        int weight = edge[1];
        int newDist = d + weight;

        if (newDist < dist[v]) {
            dist[v] = newDist;
            pq.offer(new int[]{v, newDist});
        }
    }
}

// Output results
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= N; i++) {
    if (dist[i] == INF) {
        sb.append("-1 ");
    } else {
        sb.append(dist[i]).append(" ");
    }
}
System.out.println(sb.toString().trim());
```

```
        sc.close();  
    }  
}
```

Test Case 1:

```
6 4  
1 2  
1 3  
1 4  
2 5
```

Output:

```
2 0 0 6 0 -1
```

Test Case 2:

```
5 5  
1 2  
1 3  
3 4  
4 5  
2 5
```

Output:

```
2 0 0 4 0
```

Test Case 3:

```
4 2  
1 2  
3 4
```

Output:

```
2 0 0 4
```

Test Case 4:

```
7 6  
1 2  
2 3  
3 4  
4 5  
5 6  
6 7
```

Output:

```
2 0 0 4 0 6 0
```

3)

Problem

You are organizing a new year's party and you have invited a lot of guests. Now, you want each of the guests to handshake with every other guest to make the party interactive. Your task is to know what will be the minimum time by which every guest meets others. One person can handshake with only one other person at once following the handshake should be of 3 seconds sharp.

You have prepared some data for each guest which is the order in which the $Guest(i)$ will meet others. If the data is inconsistent, then print -1. Otherwise, print the minimum time in seconds.

Hint: The data will be inconsistent if the handshake sequence of one person contradicts with another person, for instance:

```
4
2 3 4
3 1 4
4 1 2
1 3 2
```

$G(1)$ wants to meet $G(2)$ first, but $G(2)$ will meet $G(1)$ after meeting $G(3)$.

$G(3)$ will be meeting $G(2)$ after meeting $G(4)$ and $G(1)$ respectively while $G(4)$ will meet $G(3)$ after meeting $G(1)$ and $G(1)$ will meet $G(4)$ only after meeting $G(2)$ and $G(3)$. Hence, no one will be able to shake hands following this order as each handshake require some prior handshakes to happen.

Note: $G(i)$ denotes the i^{th} guest.

Input format

- The first line will contain an integer N denoting the number of guests invited.
- Next N lines will contain $N - 1$ integers where the i^{th} line will denote the sequence in which $Guest(i)$ met other guests.

Output format

If the input data is inconsistent, print -1. Otherwise, print the minimum time required for the handshakes (in seconds).

Constraints

$$1 \leq G(i) \leq N \leq 1000$$

Sample Input	Sample Output
<pre>4 2 3 4 1 3 4 4 1 2 3 1 2</pre>	12

Code:

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int minimumTimeForHandshakes(int N, vector<vector<int>>& preferences) {
    vector<vector<int>> adj(N + 1);
    vector<int> indegree(N + 1, 0);
```

```
// Build the adjacency list and calculate indegrees
for (int i = 0; i < N; i++) {
    for (int j = 0; j < preferences[i].size() - 1; j++) {
        int u = preferences[i][j];
        int v = preferences[i][j + 1];
        adj[u].push_back(v);
        indegree[v]++;
    }
}

queue<int> q;
vector<int> topological_order;
vector<int> level(N + 1, 0);

// Initialize the queue with nodes having zero indegree
for (int i = 1; i <= N; i++) {
    if (indegree[i] == 0) {
        q.push(i);
        level[i] = 1;
    }
}

// Perform topological sort and calculate levels
while (!q.empty()) {
    int u = q.front();
    q.pop();
    topological_order.push_back(u);

    for (int v : adj[u]) {
        indegree[v]--;
        if (indegree[v] == 0) {
            q.push(v);
            level[v] = level[u] + 1;
        }
    }
}

// Check for cycles (inconsistent data)
if (topological_order.size() != N) {
    return 0; // Return 0 for inconsistent data
}

// Find the maximum level
int max_level = 0;
for (int i = 1; i <= N; i++) {
    max_level = max(max_level, level[i]);
}

return max_level * 3; // Each level takes 3 seconds
```

```
}

int main() {
    int N;
    cout << "Enter the number of guests: ";
    cin >> N;

    vector<vector<int>> preferences(N);

    cout << "Enter the handshake sequence for each guest (0 to terminate input for a guest):\n";
    for (int i = 0; i < N; i++) {
        while (true) {
            int guest;
            cin >> guest;
            if (guest == 0) break; // 0 indicates the end of input for this guest
            preferences[i].push_back(guest);
        }
    }

    int result = minimumTimeForHandshakes(N, preferences);

    if (result == 0) {
        cout << "Inconsistent handshake data\n";
    } else {
        cout << "Minimum time required: " << result << " seconds\n";
    }

    return 0;
}
```

Test Case 1:

Enter the number of guests: 2

Enter the handshake sequence for each guest (0 to terminate input for a guest):

1 0

2 0

Minimum time required: 3 seconds

Test Case 2 :

Enter the number of guests: 3

Enter the handshake sequence for each guest (0 to terminate input for a guest):

1 2 3 0

2 1 0

3 1 0

Inconsistent handshake data

4)

Let's consider some weird country with **N** cities and **M** bidirectional roads of 3 types. It's weird because of some unusual rules about using these roads: men can use roads of types **1** and **3** only and women can use roads of types **2** and **3** only. Please answer the following very interesting question: what is maximum number of roads it's possible to destroy that the country will be still connected for both men and women? Connected country is country where it's possible to travel from any city to any other using existing roads.

Input

The first line contains 2 space-separated integer: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from **1** to **N** each and denote numbers of vertices that are connected by this edge. **c** denotes type of this edge.

Output

For each test case output one integer - maximal number of roads it's possible to destroy or -1 if the country is not connected initially for both men and women.

Constraints

- $1 \leq N \leq 1000$
- $1 \leq M \leq 10\,000$
- $1 \leq a, b \leq N$
- $1 \leq c \leq 3$

Sample Input	Sample Output
<pre> 5 7 1 2 3 2 3 3 3 4 3 5 3 2 5 4 1 5 2 2 1 5 1 </pre>	<pre> 2 </pre>

Idea:

This is a minimum spanning tree (MST) problem, where:

1. We need to create two separate spanning trees:

- o One for men using roads of type 1 and 3.
- o One for women using roads of type 2 and 3.

2. We also track the overall connectedness by combining roads of type 3 in both trees to maximize the number of roads we can remove.

Code:

```

#include <iostream>
#include <vector>
#include <tuple>

```

```
using namespace std;

class DisjointSet
{
public:
    vector<int> parent, rank;

    DisjointSet(int n)
    {
        parent.resize(n + 1);
        rank.resize(n + 1, 0);
        for (int i = 1; i <= n; i++)
            parent[i] = i;
    }

    int find(int u)
    {
        if (parent[u] != u)
            parent[u] = find(parent[u]);
        return parent[u];
    }

    bool unite(int u, int v)
    {
        int rootU = find(u), rootV = find(v);
        if (rootU == rootV)
            return false;
        if (rank[rootU] > rank[rootV])
        {
            parent[rootV] = rootU;
        }
        else if (rank[rootU] < rank[rootV])
        {
            parent[rootU] = rootV;
        }
        else
        {
            parent[rootV] = rootU;
            rank[rootU]++;
        }
        return true;
    }
};

int main()
{
    int N, M;
    cout << "Enter the number of cities (N) and roads (M): ";
    cin >> N >> M;
```

```
vector<tuple<int, int, int>> edges;
cout << "Enter the roads (u, v, type):" << endl;
for (int i = 0; i < M; i++)
{
    int u, v, type;
    cin >> u >> v >> type;
    edges.push_back(make_tuple(u, v, type));
}

DisjointSet men(N), women(N);

int totalEdges = 0;
int usedEdgesMen = 0, usedEdgesWomen = 0;

// Process type 3 edges first (both men and women)
for (auto [u, v, type] : edges)
{
    if (type == 3)
    {
        bool connectedMen = men.unite(u, v);
        bool connectedWomen = women.unite(u, v);
        if (connectedMen || connectedWomen)
        {
            totalEdges++;
            if (connectedMen)
                usedEdgesMen++;
            if (connectedWomen)
                usedEdgesWomen++;
        }
    }
}

// Process type 1 (men) and type 2 (women) edges
for (auto [u, v, type] : edges)
{
    if (type == 1)
    {
        if (men.unite(u, v))
        {
            totalEdges++;
            usedEdgesMen++;
        }
    }
    else if (type == 2)
    {
        if (women.unite(u, v))
        {
            totalEdges++;
            usedEdgesWomen++;
        }
    }
}
```

```
    }  
}  
  
// Check if all cities are connected for men and women  
bool fullyConnectedMen = true, fullyConnectedWomen = true;  
for (int i = 1; i <= N; i++)  
{  
    if (men.find(i) != men.find(1))  
        fullyConnectedMen = false;  
    if (women.find(i) != women.find(1))  
        fullyConnectedWomen = false;  
}  
  
// Output the result  
if (fullyConnectedMen && fullyConnectedWomen)  
{  
    cout << "Maximum number of roads that can be destroyed: " << M - totalEdges <<  
endl;  
}  
else  
{  
    cout << "-1" << endl;  
}  
  
return 0;  
}
```

Test Case 1:

Enter the number of cities (N) and roads (M): 5 7

Enter the roads (u, v, type):

1 2 3

2 3 3

3 4 3

5 3 2

5 4 1

5 2 2

1 5 1

Maximum number of roads that can be destroyed: 2

Test Case 2:

Enter the number of cities (N) and roads (M): 3 3

Enter the roads (u, v, type):

1 2 1

2 2 3

3 1 2

-1

5)

Telecom towers are an integral part of the telecom network infrastructure. In fact they are the most expensive to build and the valuations are heavy. The newly started mobile company in Hacker Land built n towers to enhance the connectivity of their users. You can assume that the **Cartesian coordinate system** is used in Hacker Land and the location of i^{th} tower is given as (x_i, y_i) .

After the construction of the towers company realised that there are many call drops happening with the users. One identified reason for the frequent call drop was that the pair of towers which are at Euclidean distance d were causing destructive interference. To resolve the issue the company decided to destroy some towers such that no two towers are at d distance. You have to tell the minimum number of towers that the company need to destroy such that no two towers are at distance d .

Note: The Euclidean distance between points (x_1, y_1) and (x_2, y_2) is the length of the line segment connecting them, which is same as $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Constraints:

- $2 \leq n \leq 10^4$
- $1 \leq d \leq 200$
- $1 \leq x_i, y_i \leq 200$
- There are no two towers with same location.

Input Format:

The first line contains two space-separated integers n and d denoting the number telecom towers constructed initially and the distance which causes destructive interference respectively.

Next n lines contains two-space separated integers denoting the x_i and y_i — location of the towers.

Output Format:

Print a single lines denoting the minimum number of towers that should be destroyed such that no two towers are separated by distance d .

Sample Input	Sample Output
5 2 1 3 3 1 3 3 3 5 5 3	1

Code:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;
```

```
struct Tower
{
    int x, y;
};

bool canDestroy(const vector<Tower> &towers, int d, int k)
{
    vector<bool> destroyed(towers.size(), false);
    int count = 0;

    for (int i = 0; i < towers.size(); i++)
    {
        if (destroyed[i])
            continue;
        count++;
        for (int j = i + 1; j < towers.size(); j++)
        {
            if (destroyed[j])
                continue;
            int dx = towers[i].x - towers[j].x;
            int dy = towers[i].y - towers[j].y;
            if (dx * dx + dy * dy <= d * d)
            {
                destroyed[j] = true;
            }
        }
    }

    return towers.size() - count <= k;
}

int minTowersToDestroy(const vector<Tower> &towers, int d)
{
    int left = 0, right = towers.size();
    while (left < right)
    {
        int mid = left + (right - left) / 2;
        if (canDestroy(towers, d, mid))
        {
            right = mid;
        }
        else
        {
            left = mid + 1;
        }
    }
    return left;
}
```

```
int main()
{
    int n, d;
    cout << "Enter the number of towers (n) and the interference distance (d): ";
    cin >> n >> d;

    vector<Tower> towers(n);
    cout << "Enter the coordinates of each tower (x y):" << endl;
    for (int i = 0; i < n; i++)
    {
        cout << "Tower " << i + 1 << ": ";
        cin >> towers[i].x >> towers[i].y;
    }

    int result = minTowersToDestroy(towers, d);
    cout << "Minimum number of towers to destroy: " << result << endl;

    return 0;
}
```

Test Case 1:

```
Enter the number of towers (n) and the interference distance (d): 5 2
Enter the coordinates of each tower (x y):
Tower 1: 1 3
Tower 2: 3 1
Tower 3: 3 3
Tower 4: 3 5
Tower 5: 5 3
Minimum number of towers to destroy: 1
```

Test Case 2:

```
Enter the number of towers (n) and the interference distance (d): 4 2
Enter the coordinates of each tower (x y):
Tower 1: 1 1
Tower 2: 1 3
Tower 3: 3 1
Tower 4: 3 3
Minimum number of towers to destroy: 2
```